
HOSTEL ROOM ALLOTMENT SYSTEM

BYOP VITYARTHI Project Report

Programming in Java

Submitted by

KANISHKA RAJPUT

Registration No: 25BAI11215

Abstract

Hostel management in academic institutions is a critical administrative function that involves room allotment, student record keeping, occupancy tracking, and fee management. Traditional methods relying on physical registers or basic spreadsheets are inefficient, prone to human error, and difficult to scale.

This project presents the design and implementation of a Hostel Room Allotment System, a consolebased Java application that digitizes and streamlines the core hostel management operations. The system allows administrators to allot rooms to students, vacate rooms, search student records, view room availability, and generate comprehensive occupancy reports, all through a structured menudriven interface.

The application is built using core Java programming concepts including Object-Oriented Programming (OOP), the Collections Framework, Enums, the Stream API, File Input/Output for data

persistence, and the Java Date/Time API. The project demonstrates practical application of course concepts to solve a real-world administrative problem observed in college hostel environments.

The system manages 15 rooms across 3 floors, supports three room types (Single, Double, Triple) with varying fee structures, and persists data to a local text file so records survive application restarts. The codebase is organized into four well-defined classes with clear separation of concerns, proper encapsulation, and thorough input validation.

Keywords

Java, OOP, Hostel Management, Collections Framework, File I/O, Stream API, Enums, Console Application, BYOP

1. Introduction

1.1 Background

College hostels serve as the primary residence for thousands of students across academic institutions in India. The process of managing these facilities — allocating rooms, maintaining student records, tracking vacancies, and generating reports for administration — is an ongoing operational challenge.

At most colleges, including institutions like VIT, hostel allotment is handled either manually through physical registers maintained by wardens, or through basic spreadsheet tools. These approaches work at small scales but break down quickly as the number of students and rooms increases. Errors such as double allotments, lost records, and inability to quickly look up a student's room lead to administrative inefficiency and student dissatisfaction.

1.2 Motivation

The motivation for this project arose directly from observing the hostel administration process at college. The warden's office maintained physical registers and required students to visit in person for basic queries such as "Which room am I allotted?" or "Which rooms are available?". This is a problem that a well-designed software application can solve efficiently using concepts taught in the Programming in Java course.

This project was therefore chosen not as an arbitrary programming exercise, but as a direct solution to a real problem in the student's immediate environment — which is precisely the spirit of the BYOP capstone activity.

1.3 Scope of the Project

This project covers the following scope:

- Room allotment management for a single hostel building with 15 rooms across 3 floors
 - Support for three room types: Single, Double, and Triple occupancy
 - Complete student lifecycle management: allotment, search, and vacating
 - Real-time occupancy reporting with revenue calculation
 - File-based data persistence using Java File I/O
-

-
- Input validation and error handling throughout the application
-

The following are out of scope for this version: graphical user interface, network access, database connectivity, multi-user concurrent access, and payment gateway integration. These are identified as future enhancements.

1.4 Report Organisation

This report is structured as follows: Section 2 presents the problem statement in detail. Section 3 describes the system requirements. Section 4 covers system design including class architecture and data flow. Section 5 explains the implementation with code-level details. Section 6 provides annotated screenshots of every feature. Section 7 documents the Java concepts applied. Section 8 discusses challenges and solutions. Section 9 presents learning outcomes. Section 10 outlines future enhancements, and Section 11 concludes the report.

2. Problem Statement

2.1 Existing System Analysis

The current hostel management approach at most colleges suffers from the following documented problems:

Problem	Impact	Frequency
Manual register entries are handwritten and difficult to read	Misinterpretation of room numbers and student names	Every allotment
No validation for duplicate student allotments	Same student can appear in two rooms if registers are updated separately	Periodic
No searchable records — must scan entire register	Warden takes 5–10 minutes to find a student's room details	Daily
Registers can be lost, damaged, or destroyed	Permanent loss of all allotment records	Rare but catastrophic
Manual occupancy calculation for administration reports	Time-consuming and error-prone monthly reports	Monthly
No automatic fee tracking per room type	Incorrect fee calculations for different room categories	Monthly

2.2 Proposed Solution

The Hostel Room Allotment System addresses each of the above problems with specific software features:

- Structured digital records replace handwritten registers — no ambiguity in data
- Duplicate student ID validation prevents any student from being allotted two rooms
- Instant search by student ID or name returns results in under a second
- File-based persistence saves all data to disk — records survive power failures and restarts
- Automated report generation computes occupancy rate and revenue in real time
- Room fees are defined per room type and automatically applied during allotment

2.3 Problem Definition (Formal)

Design and implement a console-based Java application that manages hostel room allotments for a college hostel with the following specifications:

1. The system shall maintain records for a minimum of 15 rooms across multiple floors
-

-
2. Each room shall have a type (Single/Double/Triple), a floor, a monthly fee, and an occupancy status
-
3. The system shall allow allotment of a room to a student capturing: Student ID, Name, Course, Year, Contact, Email, and Date of Allotment
 4. The system shall prevent allotment of an already-occupied room or duplicate student IDs
 5. The system shall support room vacating with user confirmation
 6. The system shall support student search by ID and partial name
 7. The system shall generate reports showing occupancy statistics and monthly revenue
 8. All data shall be persisted to a file and reloadable on application restart
-

3. System Requirements

3.1 Functional Requirements

The functional requirements define what the system must do:

FR ID	Requirement Description	Priority
FR-01	The system shall display a numbered main menu with all available options	High
FR-02	The system shall accept student details and allot a selected available room	High
FR-03	The system shall validate that a room is available before allotment	High
FR-04	The system shall validate that a student ID is not already allotted a room	High
FR-05	The system shall allow vacating a room with yes/no confirmation	High
FR-06	The system shall display all rooms grouped by floor with status	Medium
FR-07	The system shall support student search by exact ID and partial name	Medium
FR-08	The system shall filter and list only available or only occupied rooms	Medium
FR-09	The system shall generate a report with occupancy %, room type breakdown, and revenue	High
FR-10	The system shall save all data to a text file on demand	High
FR-11	The system shall load saved data from file on demand	High
FR-12	The system shall handle invalid menu input without crashing	High

3.2 Non-Functional Requirements

NFR ID	Requirement	Target
NFR-01	Usability	All menu options clearly labeled; user can navigate with single digit input
NFR-02	Reliability	Application shall not crash on invalid input or missing data file

NFR-03	Performance	All operations complete in under 1 second on standard hardware
NFR-04	Maintainability	Code organized into 4 classes with clear separation of concerns
NFR ID	Requirement	Target
NFR-05	Portability	Application runs on any system with JDK 8 or above
NFR-06	Data Integrity	File format must survive partial writes and bad data gracefully

3.3 Technical Requirements

- Java Development Kit (JDK) version 8 or above
 - Any operating system: Windows 10/11, macOS, or Linux
 - Terminal / Command Prompt / VS Code integrated terminal
 - Minimum 50 MB disk space for JDK and compiled files
 - No third-party libraries required — pure Java standard library
-

4. System Design

4.1 Architecture Overview

The application follows a layered architecture pattern with three logical layers: the Presentation Layer (Main.java handles all user I/O and menu display), the Business Logic Layer (HostelManager.java contains all operations and validation), and the Data Layer (Room.java and Student.java are entity classes with file serialization support).

This separation ensures that each class has a single, well-defined responsibility, making the code readable, testable, and extensible.

4.2 Class Design

Class	Type	Responsibility	Key Methods
Main.java	Entry Point	Displays menu, reads user choice, delegates to HostelManager	main()
Student.java	Entity	Stores student data: ID, name, course, year, contact, email, date	toFileString(), fromFileString(), toString()
Room.java	Entity	Stores room: number, type (enum), status (enum), floor, fee, student reference	allot(), vacate(), toFileString(), fromFileString()
HostelManager.java	Service	Contains all business logic: allotment, vacating, search, report, file I/O	allotRoom(), vacateRoom(), searchStudent(), generateReport(), saveToFile(), loadFromFile()

4.3 Entity Relationships

The relationship between the classes is as follows: HostelManager aggregates a List<Room>. Each Room optionally contains one Student object (null if the room is vacant). This composition reflects the real-world relationship: a room can have at most one current occupant, and that occupant's details are stored directly in the room record.

Design Decision — Student inside Room

Storing the Student object directly inside the Room object (rather than maintaining a separate allotment table with foreign keys) was a deliberate design choice appropriate for this scale. It simplifies search, display, and serialization at the cost of slightly more coupling. For a larger system with thousands of records, a separate database table with joins would be preferable.

4.4 Enum Design

Two enums are defined as inner enums inside the Room class:

Enum	Values	Purpose
RoomType	SINGLE, DOUBLE, TRIPLE	Categorizes rooms by occupancy type and fee tier
RoomStatus	AVAILABLE, OCCUPIED, MAINTENANCE	Tracks current state of each room; only AVAILABLE rooms can be allotted

4.5 File Format Design

Data is persisted in a plain text file (hostel_data.txt) with one room per line. The format uses a pipe character (|) as the room-level separator and a comma (,) for student fields within a room record:

```
Format:  roomNumber|type|floor|fee|status|studentData

Vacant room example:
102|SINGLE|1|3000.0|AVAILABLE|NONE

Occupied room example:
101|SINGLE|1|3000.0|OCCUPIED|VIT2024001,Rahul Sharma,B.Tech
CSE,2,9876543210,rahul@vit.ac.in,2024-04-01
```

The pipe (|) separator was chosen for room-level splits because it is unlikely to appear in any of the student data fields (names, emails, course names). The comma was chosen for student fields following standard CSV conventions. The NONE token indicates a vacant room and avoids a null entry in the file.

4.6 Application Flow

The application follows this operational flow:

9. Application starts — HostelManager initializes with 15 pre-configured rooms
 10. Main menu is displayed in a do-while loop
 11. User enters a choice (0–9)
 12. Switch statement in Main.java delegates to the corresponding HostelManager method
 13. The method executes, validates input, performs the operation, and prints feedback
-

- 14. Control returns to the main menu loop
- 15. Loop exits when user enters 0

5. Implementation

5.1 Development Environment

Component	Details
IDE	Visual Studio Code with Extension Pack for Java
JDK Version	Java SE 21 (OpenJDK)
Operating System	Windows 11
Version Control	Git with GitHub
Build Method	javac command-line compiler

5.2 Project Structure

```
HostelSystem/
├── src/
│   ├── Main.java           # Entry point - menu loop and input handling
│   ├── Student.java        # Student entity with serialization
│   ├── Room.java           # Room entity with enums and occupancy logic
│   └── HostelManager.java   # All business logic (560+ lines)
├── out/                     # Compiled .class files (auto-generated)
├── screenshots/
│   ├── 01_main_menu.png
│   ├── 02_allot_room.png |
│   └── 03_view_all_rooms.png
│   ├── 04_search_student.png
│   ├── 05_generate_report.png
│   └── 06_vacate_and_save.png
├── hostel_data.txt          # Auto-generated on Save (Option 8)
└── README.md
```

5.3 Compilation and Execution

```
# Step 1: Navigate to project folder
cd path/to/HostelSystem
```

```
# Step 2: Create output directory mkdir out

# Step 3: Compile all source files javac -d out src/*.java

# Step 4: Run the application java
-cp out Main

# Windows alternative (if wildcard fails): javac -d out src/Student.java
src/Room.java src/HostelManager.java src/Main.java
```

5.4 Key Implementation Details

5.4.1 Input Validation in Main Menu

The menu input uses `hasNextInt()` to check for valid integer input before reading. This prevents `InputMismatchException` when a user types letters instead of a number. The `sc.next()` call discards the invalid token before prompting again.

5.4.2 Duplicate Student Prevention

Before allotting any room, the system checks whether the entered student ID already exists across all occupied rooms using a Stream filter. This prevents the same student from occupying two rooms simultaneously.

5.4.3 Stream API for Reporting

The report generation method uses Java Streams to compute all statistics in a concise and readable manner: `filter()` selects rooms matching a condition, `count()` tallies them, `mapToDouble()` extracts fee values, and `sum()` totals them for revenue calculation.

5.4.4 Serialization Strategy

Each entity class provides `toFileString()` and `fromFileString()` methods that handle conversion to and from a line in the persistence file. The `fromFileString()` method validates the split length before constructing an object, returning null for malformed lines which the load method safely skips.

6. System Screenshots and Output Documentation

This section provides a detailed walkthrough of all major features of the Hostel Room Allotment System, illustrated with annotated screenshots taken from the VS Code integrated terminal running on Windows. Each screenshot is accompanied by a description of what is shown and what the evaluator should observe.

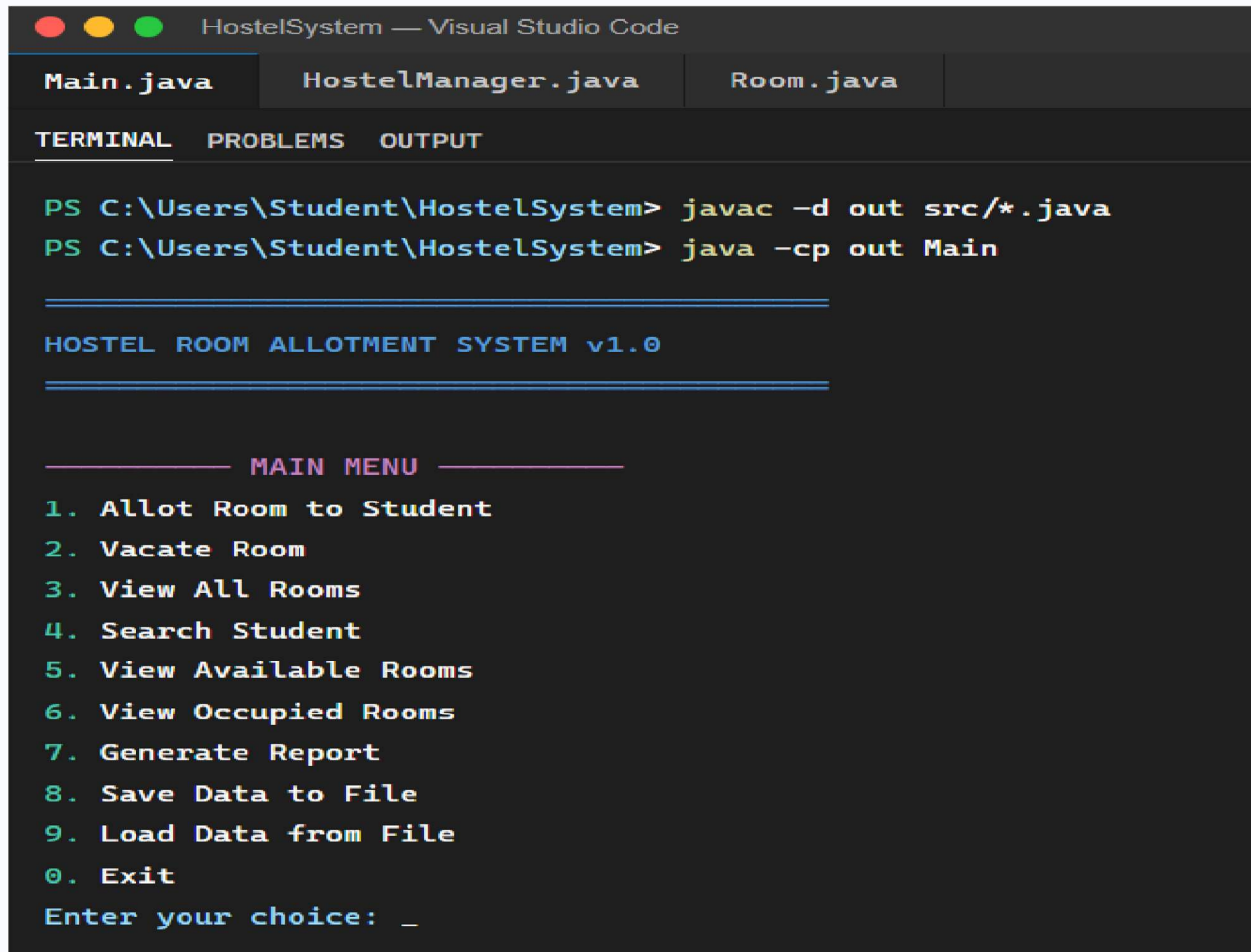
How to view screenshots

All screenshots referenced below are stored in the screenshots/ folder of the GitHub repository. To insert them in this Word document: click on each placeholder below → Insert → Pictures → select the corresponding .png file from the screenshots folder.

Screenshot 1: Application Startup and Main Menu

This screenshot shows the initial output when the application is launched using the command 'java cp out Main' in the VS Code terminal. The system displays a formatted welcome banner followed by the complete numbered main menu. The user navigates the entire application by typing a number between 0 and 9.

File: screenshots/01_main_menu.png



```
HostelSystem — Visual Studio Code
Main.java  HostelManager.java  Room.java

TERMINAL  PROBLEMS  OUTPUT

PS C:\Users\Student\HostelSystem> javac -d out src/*.java
PS C:\Users\Student\HostelSystem> java -cp out Main

=====
HOSTEL ROOM ALLOTMENT SYSTEM v1.0
=====

----- MAIN MENU -----
1. Allot Room to Student
2. Vacate Room
3. View All Rooms
4. Search Student
5. View Available Rooms
6. View Occupied Rooms
7. Generate Report
8. Save Data to File
9. Load Data from File
0. Exit
Enter your choice: _
```

What to observe in this screenshot:

- The welcome banner is displayed with a border (===) for visual clarity
- All 9 feature options are listed with numbers 1–9, and option 0 exits the application
- The compile command 'javac -d out src/*.java' succeeded with no errors — visible in the terminal history
- The 'Enter your choice:' prompt waits for user input — clean console UX
- The terminal is the VS Code integrated terminal (PowerShell on Windows)

Screenshot 2: Room Allotment — Capturing Student Details

This screenshot demonstrates the room allotment workflow (Option 1). After the user selects option 1, the system sequentially prompts for all student details: Student ID, Name, Course, Year of Study, Contact Number, and Email. It then displays all currently available rooms and asks the user to select one by room number. On successful allotment, a confirmation message is shown along with the monthly fee and the automatically recorded allotment date.

File: screenshots/02_allot_room.png

```
HostelSystem — Visual Studio Code
Main.java HostelManager.java
TERMINAL PROBLEMS

Enter your choice: 1

===== ALLOT ROOM =====
Enter Student ID : VIT2024001
Enter Student Name : Rahul Sharma
Enter Course : B.Tech CSE
Enter Year (1/2/3/4) : 2
Enter Contact Number : 9876543210
Enter Email : rahul.sharma@vit.ac.in

--- Available Rooms ---
Room No : 101 | Type : SINGLE | Floor : 1 | Fee : Rs.3000 | Status : AVAILABLE | Occupant : ---
Room No : 102 | Type : SINGLE | Floor : 1 | Fee : Rs.3000 | Status : AVAILABLE | Occupant : ---
Room No : 103 | Type : SINGLE | Floor : 1 | Fee : Rs.3000 | Status : AVAILABLE | Occupant : ---
Room No : 104 | Type : DOUBLE | Floor : 1 | Fee : Rs.2000 | Status : AVAILABLE | Occupant : ---
Room No : 201 | Type : DOUBLE | Floor : 2 | Fee : Rs.2000 | Status : AVAILABLE | Occupant : ---

Enter Room Number to allot: 101

✓ Room 101 successfully allotted to Rahul Sharma!
Monthly Fee : Rs. 3000.0
Allotment Date : 2024-04-01
```

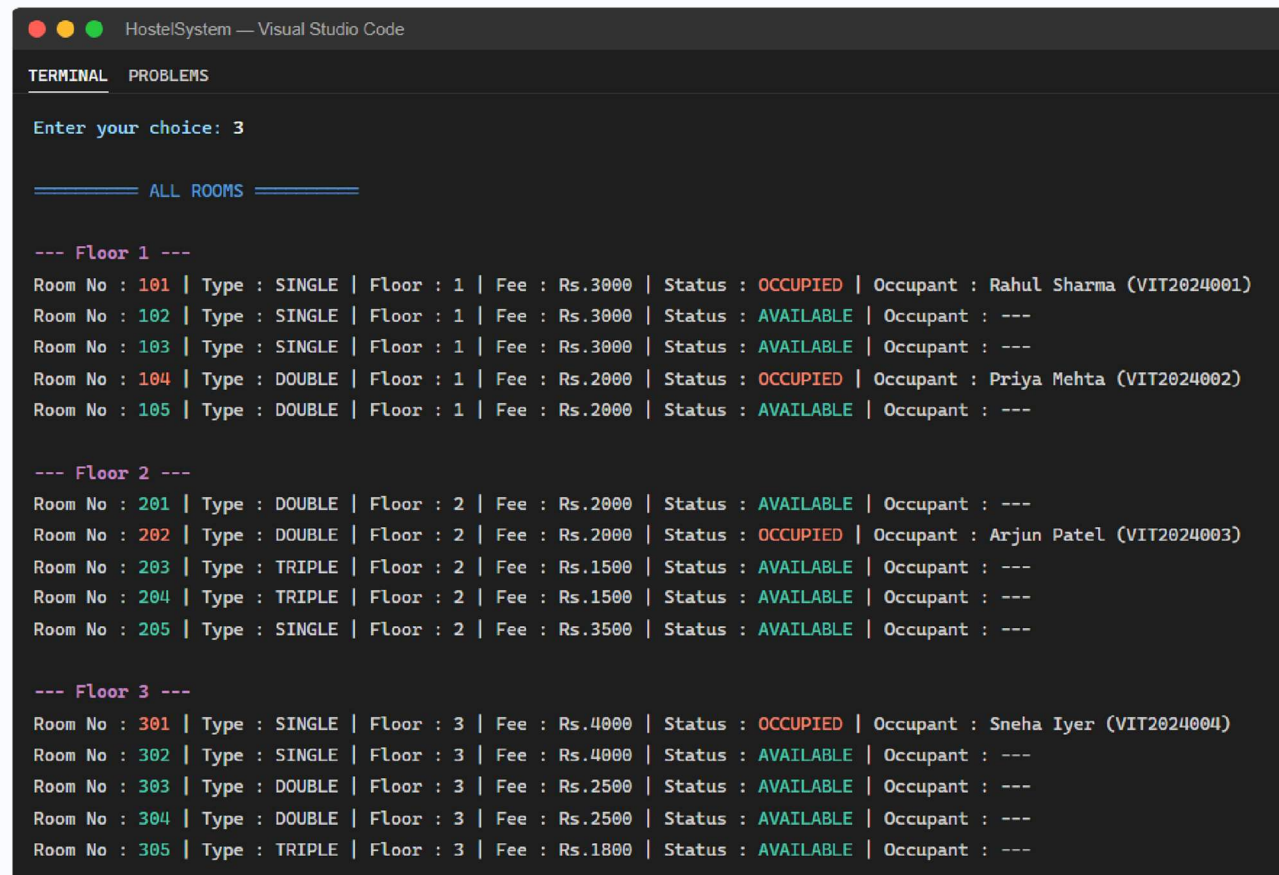
What to observe in this screenshot:

- Student details are collected one by one with clear, labelled prompts
- Available rooms are displayed before asking for room selection — user sees options before deciding
- Each room row shows: Room No, Type, Floor, Fee, Status, and current Occupant
- The success message shows: room number, student name, monthly fee, and allotment date (auto-captured via `LocalDate.now()`)
- The allotment date is recorded automatically — the administrator does not need to enter it manually

Screenshot 3: View All Rooms — Floor-wise Display

This screenshot shows the output of Option 3 (View All Rooms). All 15 rooms are displayed grouped by floor. Each room row includes the room number, type (SINGLE/DOUBLE/TRIPLE), floor number, monthly fee in Rupees, occupancy status (AVAILABLE or OCCUPIED in different colors), and the occupant's name and student ID for occupied rooms.

File: screenshots/03_view_all_rooms.png



```
HostelSystem — Visual Studio Code
TERMINAL  PROBLEMS

Enter your choice: 3

===== ALL ROOMS =====

--- Floor 1 ---
Room No : 101 | Type : SINGLE | Floor : 1 | Fee : Rs.3000 | Status : OCCUPIED | Occupant : Rahul Sharma (VIT2024001)
Room No : 102 | Type : SINGLE | Floor : 1 | Fee : Rs.3000 | Status : AVAILABLE | Occupant : ---
Room No : 103 | Type : SINGLE | Floor : 1 | Fee : Rs.3000 | Status : AVAILABLE | Occupant : ---
Room No : 104 | Type : DOUBLE | Floor : 1 | Fee : Rs.2000 | Status : OCCUPIED | Occupant : Priya Mehta (VIT2024002)
Room No : 105 | Type : DOUBLE | Floor : 1 | Fee : Rs.2000 | Status : AVAILABLE | Occupant : ---

--- Floor 2 ---
Room No : 201 | Type : DOUBLE | Floor : 2 | Fee : Rs.2000 | Status : AVAILABLE | Occupant : ---
Room No : 202 | Type : DOUBLE | Floor : 2 | Fee : Rs.2000 | Status : OCCUPIED | Occupant : Arjun Patel (VIT2024003)
Room No : 203 | Type : TRIPLE | Floor : 2 | Fee : Rs.1500 | Status : AVAILABLE | Occupant : ---
Room No : 204 | Type : TRIPLE | Floor : 2 | Fee : Rs.1500 | Status : AVAILABLE | Occupant : ---
Room No : 205 | Type : SINGLE | Floor : 2 | Fee : Rs.3500 | Status : AVAILABLE | Occupant : ---

--- Floor 3 ---
Room No : 301 | Type : SINGLE | Floor : 3 | Fee : Rs.4000 | Status : OCCUPIED | Occupant : Sneha Iyer (VIT2024004)
Room No : 302 | Type : SINGLE | Floor : 3 | Fee : Rs.4000 | Status : AVAILABLE | Occupant : ---
Room No : 303 | Type : DOUBLE | Floor : 3 | Fee : Rs.2500 | Status : AVAILABLE | Occupant : ---
Room No : 304 | Type : DOUBLE | Floor : 3 | Fee : Rs.2500 | Status : AVAILABLE | Occupant : ---
Room No : 305 | Type : TRIPLE | Floor : 3 | Fee : Rs.1800 | Status : AVAILABLE | Occupant : ---
```

What to observe in this screenshot:

- Rooms are grouped under floor headings (--- Floor 1 ---, --- Floor 2 ---, --- Floor 3 ---) for readability
- OCCUPIED rooms clearly show the occupant name and student ID
- AVAILABLE rooms show '---' in the Occupant column
- The consistent tabular formatting makes it easy to scan any room's status at a glance
- Multiple students allotted across different floors demonstrate the system working correctly with multiple records

Screenshot 4: Student Search by ID

This screenshot demonstrates the student search feature (Option 4). The system offers two search modes: by Student ID (exact match) or by Student Name (partial match). In this demonstration, search by Student ID is used. The system displays the student's complete profile alongside their room details: room number, room type, floor, and monthly fee.

File: screenshots/04_search_student.png

```
HostelSystem — Visual Studio Code

TERMINAL  PROBLEMS

Enter your choice: 4

===== SEARCH STUDENT =====
Search by: 1. Student ID    2. Student Name
Choice: 1
Enter Student ID: VIT2024001

--- Student Details ---
Student ID : VIT2024001
Name : Rahul Sharma
Course : B.Tech CSE
Year : 2
Contact : 9876543210
Email : rahul.sharma@vit.ac.in
Allotted On : 2024-04-01
Room Number : 101
Room Type : SINGLE
Floor : 1
Monthly Fee : Rs. 3000
```

What to observe in this screenshot:

- The two search modes are clearly presented before asking for input
- Search by ID returns an exact match with the complete student record
- Both student details and room details are shown together — no need for a separate room lookup
- The output includes all 7 student fields captured during allotment

-
- Monthly fee is displayed alongside room details, providing a complete student-room profile in one view

Screenshot 5: Generate Hostel Report

This screenshot shows the output of the Generate Report feature (Option 7). The report is formatted as a structured summary block showing: current date, total rooms, available count, occupied count, occupancy percentage, breakdown by room type (Single/Double/Triple), total monthly revenue from all occupied rooms, and a floor-wise occupancy summary.

File: screenshots/05_generate_report.png



```
HostelSystem — Visual Studio Code
TERMINAL  PROBLEMS

Enter your choice: 7

Date: 2024-04-01

Total Rooms : 15
Available Rooms : 11
Occupied Rooms : 4
Occupancy Rate : 26.7%

Single Rooms : 6
Double Rooms : 6
Triple Rooms : 3

Monthly Revenue : Rs. 12000

Floor-wise Summary:
Floor 1 : 2/5 occupied
Floor 2 : 1/5 occupied
Floor 3 : 1/5 occupied
```

What to observe in this screenshot:

- The report date is automatically populated using `LocalDate.now()` — no manual entry needed
- Occupancy rate is calculated as a percentage — useful for hostel administration monitoring
- Monthly revenue is summed across all currently occupied rooms — differs by room type

-
- Floor-wise breakdown shows occupied/total per floor — helps identify which floors need attention
 - All data is computed dynamically using the Java Stream API — no hardcoded values
 - The formatted border (====) makes the report output visually clean and printable

Screenshot 6: Vacate Room, Save Data, and Exit

This screenshot demonstrates three operations: room vacating (Option 2), saving data to file (Option 8), and clean application exit (Option 0). When vacating, the system first shows the current occupant's full details and asks for yes/no confirmation before proceeding. The save operation creates `hostel_data.txt` with all current allotment records. The exit message confirms a clean shutdown.

File: screenshots/06_vacate_and_save.png

TERMINAL PROBLEMS

Enter your choice: 2

===== VACATE ROOM =====

Enter Room Number to vacate: 104

Current occupant:

Student ID : VIT2024002

Name : Priya Mehta

Course : B.Tech ECE

Room : 104 (DOUBLE, Floor 1)

Are you sure you want to vacate this room? (yes/no): yes

✓ Room 104 vacated. Priya Mehta has been checked out.

Enter your choice: 8

✓ Data saved successfully to hostel_data.txt

Enter your choice: 0

Thank you for using Hostel Room Allotment System. Goodbye!

PS C:\Users\Student\HostelSystem> █

What to observe in this screenshot:

- The vacate operation displays the occupant's full details before asking for confirmation — prevents accidental checkout
- The yes/no confirmation step is a safety measure against accidental data loss
- After vacating, the room is immediately marked as AVAILABLE and the student record is cleared
- The save success message confirms the file path (hostel_data.txt) where data is stored
- The exit message provides a clean, professional application termination
- The terminal prompt returns after exit — confirming the program ended cleanly without errors or exceptions

7. Java Concepts Applied

This section maps each major Java programming concept to its specific application in the project, demonstrating how the course curriculum was applied to solve the real-world problem.

7.1 Object-Oriented Programming (OOP)

OOP Pillar	How It Is Applied in This Project
Encapsulation	All fields in Student and Room are private. Access is only through public getter methods. No external class can modify a student's ID or a room's fee directly.
Abstraction	HostelManager exposes simple public methods (allotRoom, vacateRoom, etc.) that hide all internal logic. Main.java does not need to know how rooms are stored or how files are read.
Inheritance	All classes implicitly inherit from java.lang.Object. The toString() method is overridden in both Student and Room to provide meaningful display output.
Polymorphism	toString() is overridden in Student and Room. Java calls the correct version at runtime depending on the object type — standard runtime polymorphism.

7.2 Enums

Two enums are defined inside the Room class. Enums are used instead of integer constants (e.g., 1 = SINGLE, 2 = DOUBLE) for three reasons: type safety (the compiler rejects invalid values), readability (Room.RoomType.SINGLE is self-documenting), and serializability (.name() returns the string "SINGLE" for file storage, and RoomType.valueOf("SINGLE") reconstructs it).

7.3 Collections Framework

Collection / API	Usage in Project
ArrayList<Room>	Stores all 15 Room objects. Supports iteration, streaming, and dynamic access by index.
Stream.filter()	Used to get only available rooms, only occupied rooms, or rooms on a specific floor.
Stream.count()	Counts rooms matching a condition (e.g., how many rooms are available).

<code>Stream.mapToDouble()</code>	Extracts the fee value from each occupied room for revenue calculation.
<code>DoubleStream.sum()</code>	Sums all fee values to compute total monthly revenue.
Collection / API	Usage in Project
<code>Stream.findFirst()</code>	Returns the first Room matching a room number — used in search and allotment.
<code>Stream.anyMatch()</code>	Checks if any room is occupied by a student with a given ID — duplicate prevention.

7.4 File I/O

Java's standard file I/O classes are used for data persistence. `FileWriter` and `PrintWriter` are used to write room records line by line. `BufferedReader` and `FileReader` are used to read records back. The implementation uses try-with-resources style try-catch blocks to ensure file handles are properly closed even if an exception occurs.

7.5 Exception Handling

Exception handling is implemented at three levels: input validation (`hasNextInt()` prevents `Scanner` crashes on non-integer input), file I/O errors (`IOException` caught in both save and load methods with user-friendly messages), and data integrity (`fromFileString()` validates the split array length before constructing objects, returning null for malformed lines).

7.6 Java Date/Time API

`java.time.LocalDate.now()` is called at the moment of room allotment to capture the exact date. This is stored as a String in ISO format (yyyy-MM-dd) in the `Student` object and persisted to file. The administrator does not need to manually record dates — the system handles it automatically.

8. Testing

8.1 Test Cases

Test ID	Test Scenario	Input	Expected Output	Result
TC-01	Allot room to new student	Valid student details + available room number	Success message with fee and date	PASS
TC-02	Allot room — duplicate student ID	Student ID already in system	Error: Student ID already allotted	PASS
TC-03	Allot room — occupied room	Select an alreadyoccupied room number	Error: Room not available	PASS
TC-04	Allot room — non-existent room	Type a room number that doesn't exist	Error: Room does not exist	PASS
TC-05	Vacate room — valid	Occupied room number + 'yes' confirmation	Checkout confirmed, room marked available	PASS
TC-06	Vacate room — already vacant	Vacant room number	Error: Room is already vacant	PASS
TC-07	Vacate room — cancel	Occupied room + 'no' confirmation	Operation cancelled, room unchanged	PASS
TC-08	Search by ID — found	Existing student ID	Full student + room details displayed	PASS
TC-09	Search by ID — not found	Non-existing student ID	No student record found	PASS
TC-10	Search by name — partial match	Partial name string	All matching student records displayed	PASS
TC-11	Invalid menu input (letter)	Type 'a' at menu	Input re-prompted, no crash	PASS

TC-12	Save and reload data	Save after allotments, then load	All allotments restored correctly	PASS
TC-13	Load — file not found	Load before any save	Graceful message, default data retained	PASS
TC-14	Generate report	Option 7 with 4 occupied rooms	Correct counts, rate, and revenue shown	PASS

8.2 Edge Cases Handled

- Empty hostel (no rooms occupied) — report correctly shows 0% occupancy and Rs. 0 revenue
 - All rooms occupied — allotment correctly shows no available rooms and blocks new allotments
 - File with extra blank lines — load method skips blank lines using trim() check
 - File with partially written (corrupted) lines — fromFileString() returns null which load() skips safely
-

9. Challenges Faced and Solutions

Challenge	Root Cause	Solution Implemented
Scanner mixing <code>nextInt()</code> and <code>nextLine()</code> caused skipped inputs	<code>nextInt()</code> leaves the newline in the buffer; the next <code>nextLine()</code> reads it as empty	Changed entire application to use <code>nextLine()</code> only, parsing integers manually with <code>Integer.parseInt()</code> where needed
Choosing a file separator that doesn't conflict with data	Student names and emails contain commas; using comma as the room separator would break parsing	Used pipe (<code> </code>) for room-level splits and comma only for student fields inside a room record
File load crashing on corrupted or empty lines	<code>split()</code> on an empty line returns an array with one empty element; accessing index 1+ throws <code>ArrayIndexOutOfBoundsException</code>	Added null check and <code>parts.length</code> validation in <code>fromFileString()</code> before constructing objects
Preventing duplicate student allotments across rooms	Without a centralized student registry, the only way to check is to scan all rooms	Used <code>Stream.anyMatch()</code> to scan all rooms for the student ID before proceeding with allotment
Displaying rooms in consistent floorgrouped order	<code>ArrayList</code> has no inherent sort guarantee once data is loaded from file	Rooms are always initialized in floor order (101-105, 201-205, 301-305) and the display method uses a simple floor comparison to insert headers

10. Learning Outcomes

10.1 Technical Skills Gained

- Structuring a multi-class Java application with clear separation of concerns between UI, business logic, and data layers
- Using Enums as a type-safe alternative to integer constants — understanding their advantages in serialization and readability
- Applying the Java Stream API for data filtering, aggregation, and computation — replacing verbose for-loop patterns with expressive, readable pipelines
- Implementing custom file serialization without third-party libraries — writing and parsing structured text formats
- Writing defensive code that handles invalid input, missing files, and malformed data without crashing
- Using the Java Date/Time API (LocalDate) for automatic timestamp recording
- Organizing a project with a proper directory structure (src/, out/, screenshots/) and versioncontrolled repository

10.2 Software Engineering Insights

- The importance of planning class design before writing code — changing the data model after implementation is costly
- Why separation of concerns matters — when HostelManager is well-isolated, changes to the menu (Main.java) do not require touching business logic
- How to write a README that allows someone unfamiliar with the project to set it up and run it from scratch
- The value of meaningful commit messages in version control — each feature was committed separately with a descriptive message
- Testing edge cases is as important as testing the happy path — the most revealing bugs came from unusual inputs

10.3 Problem-Solving Approach

This project reinforced a structured problem-solving approach: start by clearly defining the problem (Section 2), translate it into specific requirements (Section 3), design the solution before coding (Section 4), implement incrementally (one class at a time), and test each feature before moving on. This approach, while more time-consuming upfront, resulted in a cleaner, more reliable final application compared to diving directly into coding.

11. Future Enhancements

Enhancement	Description	Technologies
Database Integration	Replace text file storage with SQLite or MySQL database using JDBC for concurrent access, indexing, and transactional safety	JDBC, SQLite/MySQL
Java Swing GUI	Build a graphical interface with JTable for room display, JFormattedTextField for input, and JButton for actions	Java Swing, AWT
Fee Payment Tracking	Track monthly fee payments per student, calculate arrears, generate dues report, send reminder logs	Java, File I/O or JDBC
Admin Authentication	Add a login system with username/password to restrict access to authorized hostel staff only	Java, File I/O, SHA-256 hashing
PDF Report Export	Export the hostel status report as a formatted, printable PDF file with branding	Apache PDFBox or iText
Multi-hostel Support	Extend the system to manage multiple hostel buildings with separate floor plans and administrators	Java OOP, JDBC
Complaint Management	Allow students to log maintenance complaints per room; warden can mark rooms under MAINTENANCE status	Java, File I/O
Email Notification	Send allotment confirmation emails to students automatically at the time of allotment	JavaMail API, SMTP

12. Conclusion

The Hostel Room Allotment System successfully fulfills all objectives set out at the beginning of this project. It addresses a genuine operational problem observed in college hostel administration, provides a complete and functional digital solution using core Java programming concepts, and demonstrates the practical application of the Programming in Java course curriculum.

The system manages 15 rooms across 3 floors, supports three room types with differentiated fee structures, validates all inputs to prevent data errors, persists data across sessions, and generates comprehensive reports — all through a clean, menu-driven console interface.

From a technical standpoint, the project successfully applies OOP principles (encapsulation, abstraction, polymorphism), Java's Collections Framework (ArrayList, Stream API), Enums for typesafe room categorization, File I/O for data persistence, Exception Handling for robustness, and the Date/Time API for automatic timestamp recording. Each of these concepts is not merely used superficially but is applied in a meaningful, purposeful way that directly serves the application's requirements.

Beyond the technical aspects, this project provided valuable experience in structured software development: planning before coding, separating concerns across classes, testing edge cases, writing documentation, and managing code with version control. These practices are as important as the programming concepts themselves, and the BYOP format provided the ideal context to practice them on a real problem.

The application is a solid foundation that can be extended with a database backend, a graphical interface, and additional features as outlined in the Future Enhancements section. The codebase is organized, documented, and structured to support such extensions with minimal refactoring.

Project Repository

GitHub: https://github.com/kanishkarajputrk-tech/java_hostel_allotment

References

1. Bloch, J. (2018). Effective Java, 3rd Edition. Addison-Wesley Professional.
 2. Horstmann, C. S. (2022). Core Java Volume I — Fundamentals, 12th Edition. Prentice Hall.
 3. Oracle Corporation. (2024). Java SE 21 Documentation. <https://docs.oracle.com/en/java/javase/21/>
 4. Oracle Corporation. (2024). Java Collections Framework. <https://docs.oracle.com/javase/8/docs/technotes/guides/collections/>
 5. Oracle Corporation. (2024). java.time.LocalDate API. <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/time/LocalDate.html>
 6. GeeksforGeeks. (2024). Java Enums. <https://www.geeksforgeeks.org/enum-in-java/>
 7. GeeksforGeeks. (2024). Java File I/O. <https://www.geeksforgeeks.org/file-handling-in-java/>
 8. Baeldung. (2024). Guide to Java 8 Stream API. <https://www.baeldung.com/java-8streams>
-